

Implementação e Análise de Desempenho da Simulação de N-Corpos em GPU utilizando CUDA

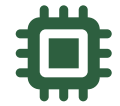
Bruno A. M. de Abreu Filho

Emily I. S. Hamano · Pedro Costa Freitas

Instituto Federal Goiano — Campus Iporá (IF GOIANO)

2025

| Introdução



Arquiteturas Paralelas

GPUs processam grandes volumes de dados com alto paralelismo



Objetivo

Implementar e analisar desempenho da simulação com CUDA



Três Versões

CPU sequencial, GPU global e GPU shared memory

Comparar tempo de execução e speedup entre CPU e GPU para diferentes quantidades de partículas

I O Problema de N-Corpos

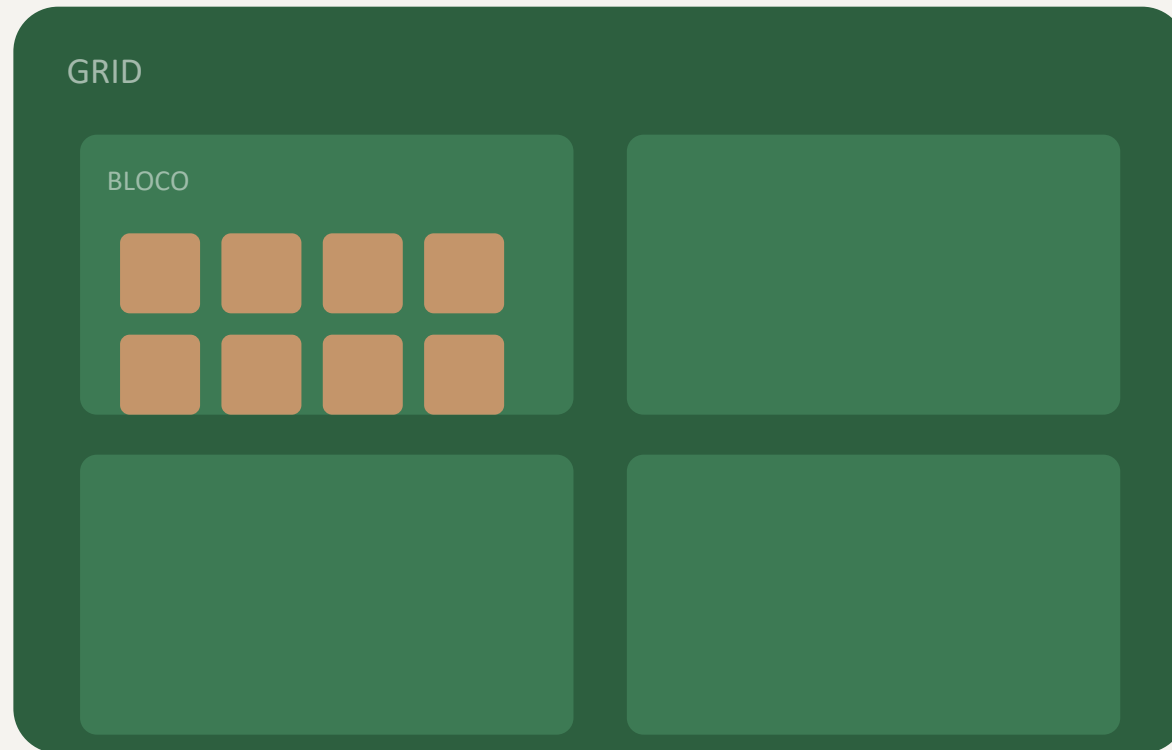
COMPLEXIDADE COMPUTACIONAL

$O(N^2)$

Cada corpo interage com todos os demais

- 🌐 Simulação de interações gravitacionais entre múltiplos corpos físicos
- 📈 Custo computacional cresce quadraticamente com o número de partículas
- ⚡ Candidato ideal para paralelização — cálculo das forças é independente
- ★ Aplicações: astronomia, física computacional, dinâmica molecular

I CUDA e Paralelismo



Grid → Blocos → Threads (modelo SIMT)



Threads — menores unidades de execução, agrupadas em blocos



Blocos — formam a grid que executa o kernel na GPU



Warp — grupo de threads executadas simultaneamente



Milhares de threads executam em paralelo sobre múltiplos dados (SIMT)

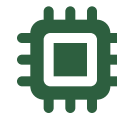
I Implementação: CPU vs GPU

CPU Sequencial



- Laços aninhados $O(N^2)$
- Execução sequencial
- Um corpo por vez
- Referência base para comparação

GPU Naive



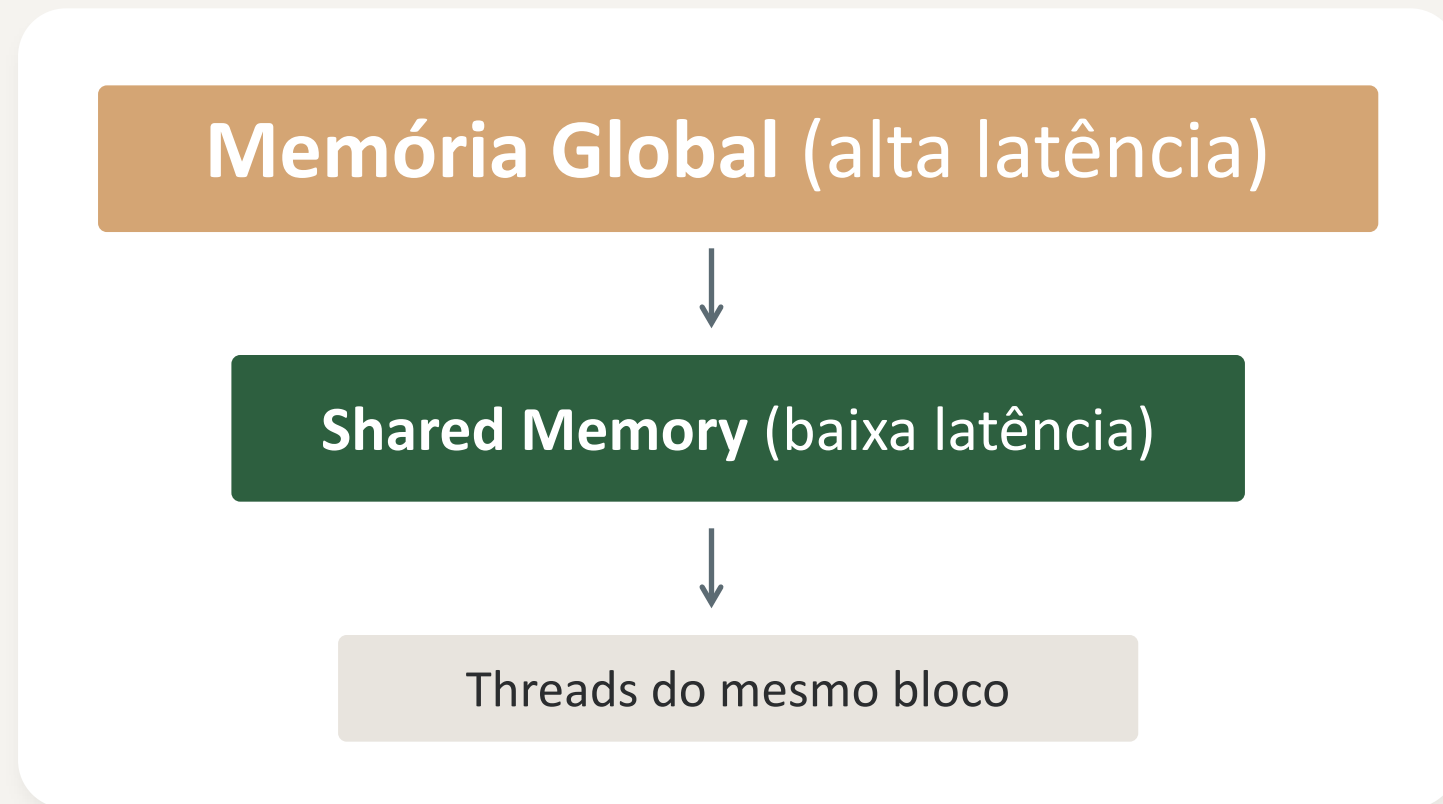
- Uma thread por corpo
- Memória global da GPU
- Paralelismo massivo
- Acesso direto à memória

GPU Shared



- Uma thread por corpo
- Memória compartilhada
- Reduz latência de acesso
- Dados reutilizados no bloco

I Shared Memory



Dados carregados temporariamente para acesso rápido

- ⚡ Memória de **alta velocidade** acessível pelas threads do bloco
- ↓ **Reduz acessos** à memória global de maior latência
- ♻️ Melhora **reutilização** de dados entre as threads
- 📈 Ganho potencial **aumenta** com o número de partículas

I Metodologia



Linguagem principal + Numba CUDA



Execução em GPU gratuita



Versionamento do código



Geração de gráficos e visualizações

Configuração dos Experimentos

Quantidades testadas

N = 100

N = 200

N = 500

Métrica de desempenho

$$\text{Speedup} = T_{\text{CPU}} / T_{\text{GPU}}$$

Posições aleatórias bidimensionais · Uma thread por corpo · Tempos médios de execução

Resultados e Benchmarks

N	CPU (s)	GPU Naive (s)	GPU Shared (s)	Speedup Naive
100	0,279	0,167	0,223	1,67x
200	1,520	0,153	0,220	9,92x
500	7,690	0,220	0,226	34,54x

SPEEDUP MÁXIMO OBTIDO

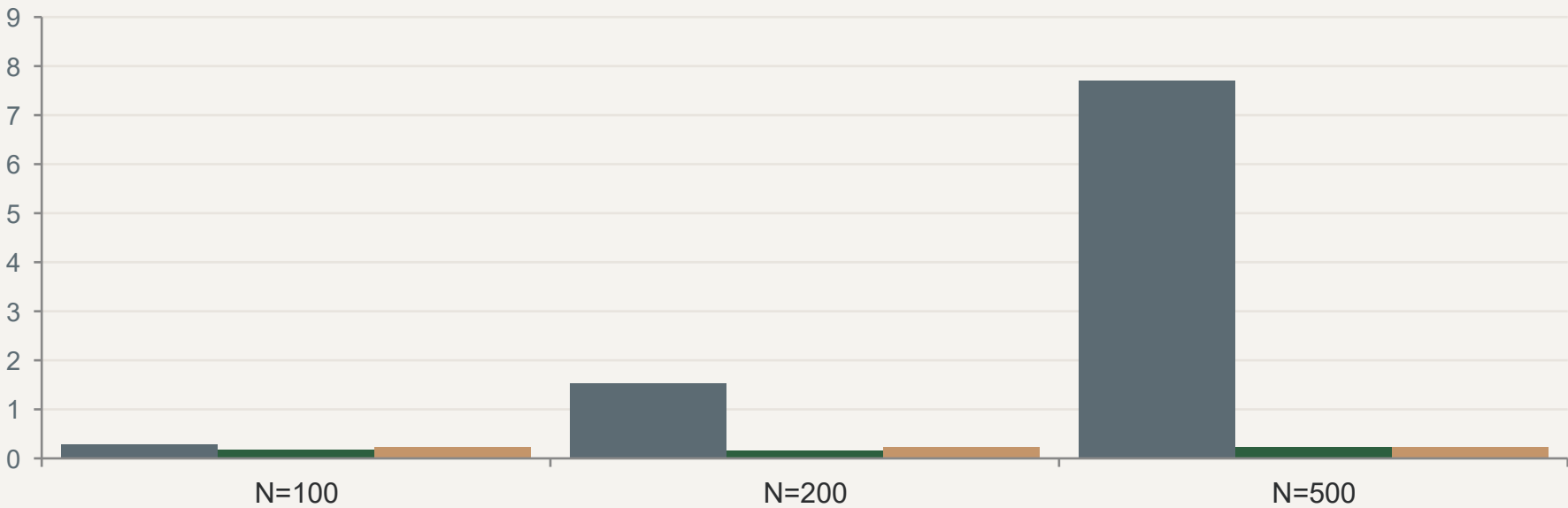
34,54x

GPU Naive vs CPU
para N = 500 partículas

Principais Observações

- Ganho cresce com N — melhor aproveitamento do paralelismo
- Para N pequeno, overhead de transferência limita speedup
- GPU Shared não superou GPU Naive nos testes
- Shared Memory tende a ser vantajosa para N maiores

CPU GPU Naive GPU Shared



I Conclusões

- 1 Ganhos **significativos** de desempenho com GPU em relação à CPU sequencial
- 2 Speedups **superiores a 30x** para os maiores conjuntos de dados avaliados
- 3 Shared Memory com **potencial** para simulações de maior escala
- 4 GPUs são **eficientes** para aplicações computacionalmente intensivas

O paralelismo em GPU demonstrou ser uma abordagem altamente eficaz para o problema de N-corpos

I Trabalhos Futuros

Ampliar Quantidade de Partículas

Testar com $N \gg 500$ para avaliar escalabilidade e ganhos da shared memory

Configurações de Blocos e Threads

Investigar diferentes tamanhos de bloco e organizações de threads na GPU

Técnicas Adicionais de Otimização

Explorar uso de memória constante, textura e outras otimizações CUDA

Algoritmo Barnes-Hut

Reduzir complexidade de $O(N^2)$ para $O(N \log N)$ com aproximação hierárquica

Obrigado!